

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación

Carrera de Software
Asignatura: Aplicaciones Web

BIOPET

Revisión cruzada del PFC seleccionado

Informe de la GA de Unidad IV

Equipo H

Mariscal Cabrera Jaime Josué
Carvajal Loor Johan Stalin
Fajardo Montes Michael Xavier

El PFC BIOPET, objeto de esta revisión, fue desarrollado por su equipo original (Mariscal Cabrera Jaime Josué, Beltrán Montiel Fred Adrián y Taipe Mora Zaida Melissa); Jaime Mariscal integra además el Equipo H. Carvajal Loor Johan Stalin y Fajardo Montes Michael Xavier no participaron en su implementación: realizan sobre él la revisión cruzada, investigación y análisis solicitados para esta actividad.

Docente: Guerrero Ulloa Gleiston Ciceron

Período académico: 2025 – 2026 — Curso/paralelo: A

Quevedo, 10 de agosto de 2026

Resumen

Este documento presenta el informe final de la actividad de GA (grupo alterno) de la Unidad IV de la asignatura Aplicaciones Web, cuyo objetivo es realizar una revisión cruzada de un Proyecto Fin de Curso (PFC) desarrollado por otro equipo. El PFC seleccionado es **BIOPET**, un sistema web de gestión veterinaria desarrollado originalmente por Mariscal Cabrera Jaime Josué, Beltrán Montiel Fred Adrián y Taipe Mora Zaida Melissa.

El Equipo H — Mariscal Cabrera Jaime Josué, Carvajal Loor Johan Stalin y Fajardo Montes Michael Xavier — recibió BIOPET ya desarrollado y funcional, y realizó sobre él: análisis de su arquitectura por capas y del patrón MVC, evaluación de la API REST y su documentación Swagger/OpenAPI respaldada con pruebas automatizadas, colecciones Postman y mediciones técnicas (JaCoCo, SUS, Lighthouse); una comparación SOAP vs REST, donde SOAP se **compara** con el caso real de BIOPET sin haber sido implementado; una investigación sobre Jamstack, *Progressive Web Apps* (PWA) e inteligencia artificial generativa, **analizadas** como tendencias aplicables y no como funcionalidades existentes en el sistema; y dos revisiones críticas independientes realizadas por Carvajal y Fajardo, consolidadas en un capítulo de revisión cruzada que identificó fortalezas verificables junto con riesgos concretos — entre ellos una inconsistencia de autorización en el listado de consultas médicas, documentada como pendiente de corrección, no como resuelta. El informe cierra con conclusiones y trabajo futuro priorizado, sin declarar a BIOPET listo para producción.

Abstract

This document presents the final report for the Unit IV cross-review (GA) assignment of the Web Applications course, whose goal is to carry out a cross-team review of a Course Final Project (PFC) developed by a different team. The selected PFC is **BIOPET**, a veterinary management web system originally developed by Mariscal Cabrera Jaime Josué, Beltrán Montiel Fred Adrián, and Taípe Mora Zaida Melissa.

Equipo H — Mariscal Cabrera Jaime Josué, Carvajal Looor Johan Stalin, and Fajardo Montes Michael Xavier — received BIOPET already developed and functional, and carried out: analysis of its layered architecture and MVC pattern, evaluation of its REST API and Swagger/OpenAPI documentation backed by automated tests, Postman collections, and technical measurements (JaCoCo, SUS, Lighthouse); a SOAP vs REST comparison, where SOAP is **compared** against BIOPET's real case without having been implemented; research on Jamstack, Progressive Web Apps (PWA), and generative AI, **analyzed** as applicable trends rather than as features already existing in the system; and two independent critical reviews carried out by Carvajal and Fajardo, consolidated into a cross-review chapter that identified verifiable strengths together with concrete risks — including an authorization inconsistency in the medical consultations listing endpoint, documented as pending correction, not as resolved. The report closes with conclusions and prioritized future work, without declaring BIOPET ready for production.

Índice general

Resumen	i
Abstract	ii
1. Introducción	1
1.1. Contexto de la actividad	1
1.2. Distinción entre el equipo original del PFC y el Equipo H	1
1.3. Objetivo de este informe	2
1.4. Estado de este documento	2
2. Presentación del PFC BIOPET	3
2.1. Qué es BIOPET	3
2.2. Propósito y alcance funcional	3
2.3. Arquitectura y stack tecnológico	4
2.4. Estado del proyecto al momento de esta revisión	4
3. Modelo MVC y API REST	5
3.1. Flujo MVC de una petición autenticada	5
3.2. Endpoints de la API REST	6
3.3. Documentación OpenAPI / Swagger	7
3.4. Evidencia Postman / Newman	7
4. SOAP vs REST	8
4.1. Propósito de esta comparación	8
4.2. Qué es SOAP	8
4.3. Qué es REST	9
4.4. Comparación por criterios	9
4.5. Ejemplo práctico: la misma operación en SOAP y en REST	10
4.5.1. Enfoque SOAP (hipotético, con fines comparativos)	10
4.5.2. Enfoque REST (implementación real de BIOPET)	11
4.6. Casos de uso en el contexto ecuatoriano	12
4.7. Cuándo conviene cada enfoque	13
5. Tendencias web actuales	14
5.1. Propósito de este análisis	14
5.2. Jamstack	14
5.3. Progressive Web Apps (PWA)	15
5.4. Inteligencia artificial generativa en el desarrollo web	15
5.4.1. Privacidad, propiedad intelectual y sesgos	16

5.5. Aplicación potencial en BIOPET	16
5.5.1. Estado actual	16
5.5.2. Evolución posible	17
6. Revisión cruzada del PFC	18
6.1. Metodología de revisión cruzada	18
6.2. Fortalezas identificadas	18
6.3. Debilidades y riesgos	19
6.3.1. Inconsistencia de autorización en el listado de consultas	19
6.4. Comparación de las dos revisiones	20
6.5. Priorización de mejoras	20
6.6. Síntesis de la revisión cruzada	21
7. Resultados y evidencias	22
7.1. Pruebas automatizadas y cobertura (JaCoCo)	22
7.2. Calidad web automatizada (Lighthouse)	23
7.3. Usabilidad (SUS)	23
7.4. Seguridad (OWASP)	23
7.5. Síntesis	23
8. Conclusiones	25
9. Trabajo futuro	27
9.1. Prioridades inmediatas	27
9.1.1. Control de acceso en Consultas	27
9.1.2. Gestión de secretos	27
9.1.3. API externa	27
9.2. Mejoras de calidad y escalabilidad	28
9.2.1. Rate limiting distribuido	28
9.2.2. Cobertura de ramas	28
9.2.3. OpenAPI	28
9.3. Frontend y experiencia	28
9.3.1. Lighthouse	28
9.3.2. PWA	28
9.4. Mantenibilidad documental	29
9.5. Priorización	29

Capítulo 1

Introducción

1.1. Contexto de la actividad

Esta actividad corresponde a la modalidad de trabajo de **GA (grupo alterno)** de la Unidad IV de la asignatura Aplicaciones Web. A diferencia de las entregas anteriores del curso, en esta modalidad el proyecto técnico (PFC) no es desarrollado por el propio grupo, sino que se selecciona el PFC de otro equipo ya terminado, y el grupo GA realiza sobre él un proceso de **revisión cruzada**: análisis de arquitectura, evaluación de la API REST y su documentación, investigación de tendencias relacionadas, y retroalimentación crítica independiente por parte de cada integrante nuevo del grupo.

1.2. Distinción entre el equipo original del PFC y el Equipo H

Es importante dejar explícita, desde el inicio de este informe, una distinción que se mantiene consistentemente en todo el documento:

- **Equipo original del PFC BIOPET (equipo BMT):** Mariscal Cabrera Jaime Josué, Beltrán Montiel Fred Adrián y Taipei Mora Zaida Melissa. Este equipo diseñó, implementó y documentó técnicamente BIOPET a lo largo de las entregas previas del curso (Entrega 1A, Entrega 1B, Tercera Entrega). El código fuente, las pruebas automatizadas, la arquitectura y la evidencia técnica descrita en los capítulos 2 y 3 de este informe son obra de ese equipo.
- **Equipo H (grupo de la actividad GA):** Mariscal Cabrera Jaime Josué, Carvajal Loor Johan Stalin y Fajardo Montes Michael Xavier. Este es el grupo responsable de **este informe** y de la actividad de Unidad IV: selección del PFC, presentación, investigación comparativa, revisión crítica y consolidación de hallazgos.

Mariscal Cabrera Jaime Josué es la única persona presente en ambos equipos: participó en el desarrollo original de BIOPET y también integra el Equipo H. Carvajal Loor Johan Stalin y Fajardo Montes Michael Xavier **no participaron en el desarrollo del PFC**: su rol en esta actividad es exclusivamente de revisión crítica independiente e investigación, nunca de implementación del código de BIOPET. Ningún aporte de código o funcionalidad del PFC debe atribuirse a Carvajal o a Fajardo en este informe.

1.3. Objetivo de este informe

El objetivo de este documento es consolidar, en un único informe académico, los siguientes componentes exigidos por la actividad de Unidad IV:

1. Presentación del PFC seleccionado (BIOPET): propósito, arquitectura y tecnologías (capítulo 2).
2. Análisis del patrón MVC y de la API REST, incluyendo su documentación Swagger/OpenAPI y la evidencia de endpoints probados (capítulo 3).
3. Investigación comparativa SOAP vs REST (capítulo 4).
4. Investigación de tendencias web actuales: Jamstack, PWA e IA generativa (capítulo 5).
5. Revisión cruzada consolidada del PFC, a partir de las revisiones independientes de Carvajal y Fajardo (capítulo 6).
6. Resultados y evidencias técnicas verificables del estado actual de BIOPET (capítulo 7).
7. Conclusiones y trabajo futuro (capítulos 8 y 9).

1.4. Estado de este documento

Esta versión del informe integra ya contenido definitivo en sus nueve capítulos: la evidencia técnica verificable de BIOPET (capítulos 2 y 3), la investigación comparativa SOAP vs REST (capítulo 4), la investigación de tendencias web (capítulo 5), la revisión cruzada consolidada de Carvajal y Fajardo (capítulo 6), los resultados y evidencias empíricas (capítulo 7), y las conclusiones y el trabajo futuro derivados de todo lo anterior (capítulos 8 y 9). Ninguno de estos capítulos conserva marcadores de contenido pendiente.

Esta versión corresponde al documento final integrado: el PDF ya fue generado y compilado con MiKTeX (`biblatex/biber`, según la cadena de compilación descrita en `docs/u4/informe/README.md`), y su resultado fue revisado y validado para la entrega de esta actividad.

Capítulo 2

Presentación del PFC BIOPET

2.1. Qué es BIOPET

BIOPET es un sistema web de gestión veterinaria, desarrollado como Proyecto Fin de Curso (PFC) de la asignatura Aplicaciones Web por su equipo original (Mariscal Cabrera Jaime Josué, Beltrán Montiel Fred Adrián y Taipe Mora Zaida Melissa; ver sección 1.2). Permite el registro y autenticación de usuarios, y la administración de mascotas, citas veterinarias, consultas (historial clínico) y vacunas, con control de acceso diferenciado por rol y, en varios recursos, por propiedad del registro.

2.2. Propósito y alcance funcional

El sistema define cuatro roles reales, verificados en el código (Rol, columna usuarios.rol): `ROLE_ADMIN`, `ROLE_VETERINARIO`, `ROLE_AUXILIAR` y `ROLE_DUENO`. Los tres primeros tienen alcance global sobre los recursos gestionados por la clínica; `ROLE_DUENO` está limitado, en la mayoría de los recursos, a consultar únicamente lo asociado a sus propias mascotas.

Los módulos funcionales actualmente implementados y verificados contra el código del backend (`Backend/src/main/java/com/biopet/controller/`) son:

- **Autenticación** (`AuthController`): registro, login, refresh y logout, mediante JWT entregado por cookies `HttpOnly/Secure/SameSite=Strict`.
- **Usuarios** (`UsuarioController`): consulta del perfil propio (`/me`) y CRUD administrativo completo, este último restringido a `ROLE_ADMIN`.
- **Mascotas** (`MascotaController`): CRUD completo con control de acceso por rol y por propiedad, más un resumen estadístico por especie respaldado por una función almacenada de PostgreSQL.
- **Citas** (`CitaController`): CRUD de citas veterinarias, asociadas a una mascota y a un veterinario asignado.
- **Consultas** (`ConsultaController`): registro de diagnóstico, tratamiento y observaciones de una atención médica, asociado cronológicamente al historial de la mascota.
- **Vacunas** (`VacunaController`): registro de vacunas aplicadas por mascota, con consulta filtrable por mascota.
- **Integración con una API externa** (`ExternalApiController`): consulta de información de una especie contra un servicio de terceros, con caché Redis (patrón *cache-aside*).

2.3. Arquitectura y stack tecnológico

BIOPET sigue una arquitectura de tres capas más caché: un frontend SPA en Angular, un backend REST en Spring Boot, una base de datos relacional PostgreSQL y una capa de caché/revocación de tokens en Redis, todo orquestado con Docker Compose.

Cuadro 2.1: Stack tecnológico de BIOPET, verificado contra el código actual.

Componente	Versión / detalle
Backend	Java 21, Spring Boot 3.2.12 (Spring Team, s.f.)
Seguridad	Spring Security 6, JWT (jjwt 0.12.6, HMAC-SHA256) (Jones et al., 2015)
Persistencia	Spring Data JPA + Hibernate, PostgreSQL 16, Flyway
Caché / revocación	Redis 7 (Spring Data Redis + Spring Cache)
Documentación de API	springdoc-openapi 2.5.0 (Swagger UI) (OpenAPI Initiative, s.f.)
Cobertura	JaCoCo 0.8.12
Frontend	Angular 17.3, TypeScript 5.4, componentes standalone (Angular Team, Google, s.f.)
Manejo uniforme de errores	ProblemDetail (RFC 7807) (Nottingham & Wilde, 2016)
Orquestación	Docker Compose, Makefile

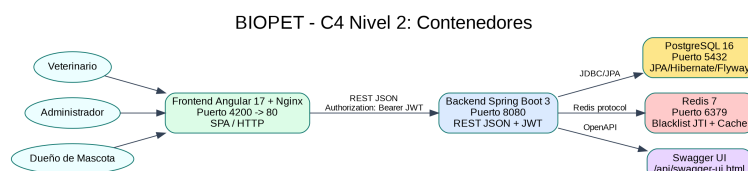


Figura 2.1: Diagrama C4 Nivel 2 (contenedores) de BIOPET.

El detalle completo de la arquitectura, con relaciones verificadas contra el código, sigue el modelo C4 (Brown, s.f.) y está documentado en `docs/diagrams/c4-contenedores/` (Nivel 2, contenedores) y `docs/diagrams/c4-componentes-backend/` (Nivel 3, componentes del backend), ambos parte del repositorio heredado del PFC.

2.4. Estado del proyecto al momento de esta revisión

Según el `README.md` raíz del repositorio, BIOPET se presenta como **versión final académica** para el alcance definido en esta actividad de Unidad IV: las funcionalidades listadas en la sección 2.2 están integradas, las pruebas automatizadas pasan (ver capítulo 7), y existe documentación técnica y evidencia reproducible respaldando cada componente. El propio README es explícito en que esto **no equivale a certificar el sistema como listo para un entorno productivo real**: usa un certificado TLS autofirmado y credenciales de desarrollo, y no fue evaluado en un entorno equivalente a producción.

Capítulo 3

Modelo MVC y API REST

3.1. Flujo MVC de una petición autenticada

BIOPET implementa el patrón Modelo-Vista-Controlador de forma estándar para una aplicación Spring Boot: el cliente Angular consume la API REST del backend, que separa controladores (**controller**), lógica de negocio (**service**) y acceso a datos (**repository/entity**). El README raíz del repositorio documenta, paso a paso y verificado contra el código, el ciclo de vida completo de una petición autenticada (**GET /api/mascotas**):

1. **Cliente Angular:** `MascotaApiService.listar()` envía la petición incluyendo la cookie de autenticación.
2. **JwtAuthenticationFilter:** intercepta la petición y resuelve el JWT desde la cookie.
3. **SecurityContextHolder:** Spring registra la autenticación del usuario ya validado.
4. **MascotaController:** recibe la petición autenticada y los parámetros de paginación.
5. **MascotaService:** ejecuta la lógica de negocio (incluye la regla de acceso por rol y por propiedad) dentro de una transacción de solo lectura.
6. **UsuarioRepository / MascotaRepository:** obtienen al usuario autenticado y, según su rol, ejecutan la consulta correspondiente (global o filtrada por dueño).
7. **PostgreSQL:** Hibernate genera y ejecuta la consulta SQL.
8. **Jackson:** serializa la respuesta a JSON.
9. **Cliente Angular:** recibe la respuesta 200 OK con el contenido paginado.

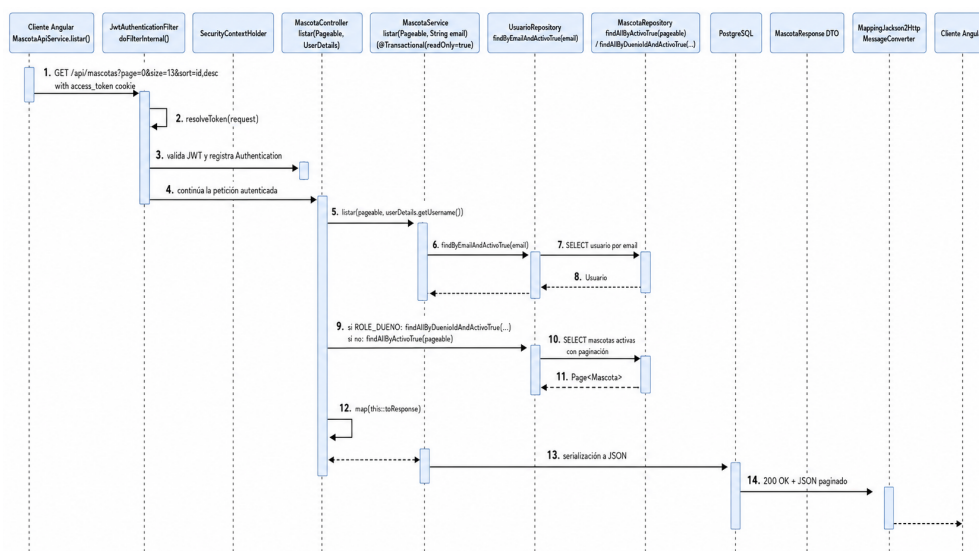


Figura 3.1: Diagrama de secuencia del flujo MVC de BIOPET (cliente Angular a PostgreSQL).

Este mismo patrón de capas (**Controller** → **Service** → **Repository**) se repite consistentemente en los demás módulos descritos en la sección 2.2 (Citas, Consultas, Vacunas, Usuarios), verificado directamente en el código fuente del backend.

3.2. Endpoints de la API REST

BIOPET expone una API REST organizada por recurso. La siguiente tabla resume los grupos de endpoints reales, verificados contra los controllers actuales del backend (Backend/src/main/java/com/biopet/controller/):

Cuadro 3.1: Recursos REST de BIOPET, verificados contra el código actual.

Recurso	Ruta base	Operaciones
Autenticación	/api/auth	registro, login, refresh, logout
Usuarios	/api/usuarios	/me + CRUD administrativo
Mascotas	/api/mascotas	CRUD + resumen por especie
Citas	/api/citas	CRUD
Consultas	/api/consultas	CRUD
Vacunas	/api/vacunas	CRUD + listado por mascota
API externa	/api/externa/especies	consulta (con caché Redis)

Todas las respuestas de error de la API usan el formato uniforme **ProblemDetail** (RFC 7807) (Nottingham & Wilde, 2016), y todos los recursos protegidos requieren la cookie de sesión emitida por /api/auth/login, con autorización adicional por rol (@PreAuthorize) y, en varios recursos, por propiedad del registro.

3.3. Documentación OpenAPI / Swagger

BIOPET documenta su API mediante `springdoc-openapi`, siguiendo la especificación OpenAPI (OpenAPI Initiative, s.f.). La configuración real (`Backend/src/main/resources/application.yml`) expone:

- **Swagger UI:** `/api/docs`.
- **Documento OpenAPI (JSON):** `/api/openapi`.

Ambas rutas son públicas (no requieren autenticación), verificado en `SecurityConfig` y probado automáticamente por `SwaggerUiTest`: la primera prueba confirma que `/api/docs` redirige a la interfaz real de Swagger UI (cuyo cuerpo contiene el marcador `swagger-ui`); la segunda confirma que `/api/openapi` responde 200 con un documento OpenAPI válido cuyo título es BIOPET API.

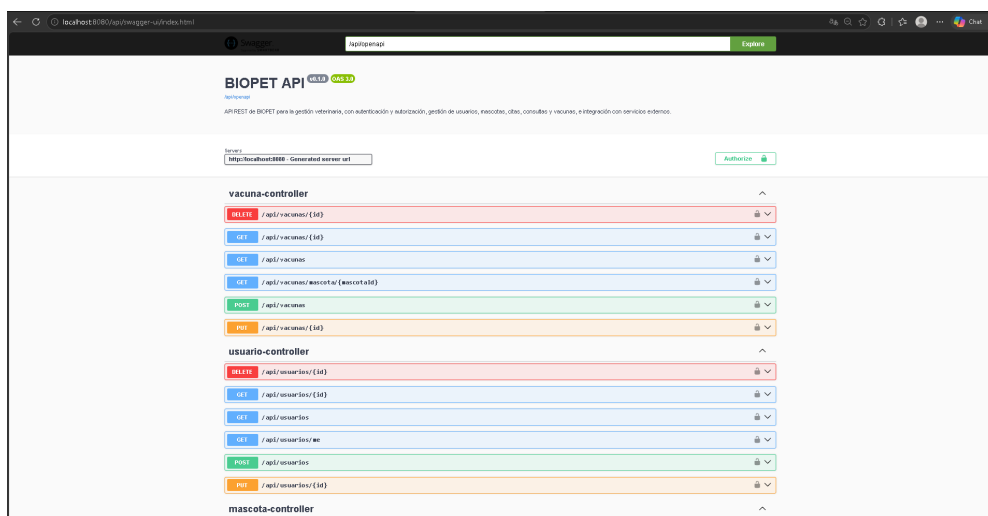


Figura 3.2: Swagger UI de BIOPET, accesible en `/api/docs`, mostrando los grupos de endpoints reales.

3.4. Evidencia Postman / Newman

El repositorio incluye colecciones Postman reales y reproducibles, verificadas contra el código actual y ejecutadas de punta a punta con Newman:

- `docs/postman/BIOPET.postman_collection.json` — colección principal (Auth + Mascotas): 44 requests definidos en 7 carpetas. Ejecución real con Newman: 46 requests, 234 assertions, 0 fallos.
- `docs/postman/BIOPET-Vacunas.postman_collection.json` — colección adicional del CRUD de Vacunas: 12 requests.

Esto demuestra, muy por encima del mínimo de tres endpoints exigido por la actividad, que la API REST de BIOPET funciona de extremo a extremo contra el backend real, incluyendo casos de éxito, de validación (422), de autenticación (401), de autorización (403) y de limitación de intentos de login (429).

Capítulo 4

SOAP vs REST

4.1. Propósito de esta comparación

La actividad de Unidad IV exige, además de la revisión cruzada del PFC seleccionado, una investigación comparativa entre los estilos SOAP y REST. Este capítulo desarrolla esa comparación tomando como referencia concreta el propio backend de BIOPET (Spring Boot, API REST documentada con OpenAPI/Swagger), de modo que la comparación no quede en el plano puramente teórico. La investigación y redacción original de este análisis corresponde a Carvajal Loor Johan Stalin, integrante del Equipo H responsable de esta línea de trabajo dentro de la GA; su aporte en esta sección es de investigación y análisis documental, no de implementación: Carvajal no participó en el desarrollo del código de BIOPET (ver sección 1.2). El contenido de este capítulo adapta y reorganiza ese aporte al formato del informe, integrando sus citas bibliográficas y precisando un detalle técnico puntual del ejemplo REST (ver observación al final de la sección 4.5).

4.2. Qué es SOAP

SOAP (*Simple Object Access Protocol*) es un protocolo de intercambio de mensajes estructurados, con especificación formal publicada por el World Wide Web Consortium (W3C) (s.f.). Su unidad de comunicación es el **sobre** (*Envelope*): un documento XML compuesto por una cabecera opcional (*Header*), que transporta metadatos de procesamiento como seguridad o enrutamiento, y un cuerpo obligatorio (*Body*), que contiene la operación invocada y sus parámetros.

El contrato entre cliente y servidor se declara mediante **WSDL** (*Web Services Description Language* (World Wide Web Consortium (W3C), 2001)): un documento XML que especifica de forma *prescriptiva* qué operaciones existen, qué tipos de datos aceptan y devuelven, y dónde están disponibles en la red. Al ser procesable por máquinas, herramientas de generación de código pueden construir automáticamente el cliente completo a partir del WSDL. Alrededor de SOAP existe además la familia de especificaciones **WS-***, entre ellas WS-Security (OASIS, 2006) para firma y cifrado a nivel de mensaje, lo que explica su vigencia en entornos corporativos con requisitos formales de seguridad y auditoría.

Conviene precisar que SOAP no está atado a HTTP: el mismo mensaje puede transportarse sobre HTTP, SMTP, JMS o una cola de mensajería, lo cual resulta valioso en arquitecturas de integración empresarial donde no todos los sistemas hablan HTTP nativamente.

4.3. Qué es REST

REST (*Representational State Transfer*) no es un protocolo sino un **estilo arquitectónico**: un conjunto de restricciones de diseño — cliente-servidor, ausencia de estado, cacheabilidad, sistema en capas, interfaz uniforme y código bajo demanda (opcional) — formuladas por R. T. Fielding (2000) al describir los principios que hicieron escalable a la propia World Wide Web.

En la práctica, una API REST modela el dominio como **recursos** identificados por URI, sobre los que se opera mediante los métodos y códigos de estado definidos por la semántica de HTTP (R. Fielding et al., 2022). El formato de representación **no está prescrito por REST**: aunque JSON se ha impuesto en la práctica por su ligereza y su soporte nativo en los navegadores, una API REST puede representar sus recursos en XML, texto plano o un formato binario sin dejar de ser REST. Es también relevante precisar que la mayoría de las API llamadas “REST” en la industria — incluida la de BIOPET — alcanzan el nivel 2 del modelo de madurez de Richardson (uso correcto de recursos y verbos HTTP), sin implementar hipermedia (HATEOAS), que sería el nivel 3 y el único que Fielding considera REST en sentido estricto.

4.4. Comparación por criterios

La tabla 4.1 resume diez criterios de comparación entre ambos enfoques, verificados contra la especificación oficial de cada uno.

Cuadro 4.1: Comparación SOAP vs REST por criterio técnico.

Criterio	SOAP	REST
Naturaleza	Protocolo con especificación W3C	Estilo arquitectónico (restricciones de diseño)
Formato de datos	XML obligatorio (<i>Envelope</i>)	Libre (JSON, XML, texto, binario); JSON predomina en la práctica
Contrato	WSDL, formal y prescriptivo (existe antes que el código)	OpenAPI, descriptivo y habitualmente generado desde el código
Transporte	Independiente: HTTP, SMTP, JMS, colas de mensajería	Ligado a la semántica de HTTP/HTTPS
Complejidad de consumo	Alta: requiere WSDL, <i>stubs</i> generados y manejo de <code>SOAPFault</code>	Baja: basta un cliente HTTP (<code>curl</code> , <code>fetch</code> , Postman)
Rendimiento y caché	Mayor sobrecarga por el sobre XML; toda operación viaja por <code>POST</code> , no cacheable	Payload más liviano; los <code>GET</code> son cacheables por la infraestructura HTTP estándar
Seguridad	WS-Security protege el <i>mensaje</i> (firma/cifrado, sobrevive a intermediarios)	TLS protege el <i>canal</i> (se descifra en cada salto)
Manejo de estado	Admite transacciones distribuidas (WS-AtomicTransaction)	<i>Stateless</i> por restricción arquitectónica
Uso empresarial típico	Banca, seguros, gobierno, sistemas legados	Web, móvil, microservicios, SaaS
Manejo de errores	Fallos viajan dentro del sobre (<code>SOAPFault</code>), típicamente bajo HTTP 500	El código de estado HTTP es parte del contrato (por ejemplo, 404)

Tres de estos diez criterios merecen desarrollo adicional porque suelen malinterpretarse. **Primero, el contrato.** El WSDL es prescriptivo: existe antes que el código y de él se genera el cliente, lo que garantiza que contrato e implementación no diverjan. OpenAPI, en cambio, cumple una función descriptiva y normalmente se produce *a partir* del código ya escrito, sin que nada obligue a que refleje fielmente el comportamiento real; en BIOPET, por ejemplo, el esquema de seguridad declarado en el documento OpenAPI (`bearerAuth`, tipo HTTP Bearer) no corresponde al mecanismo de autenticación que el sistema realmente usa en su flujo web principal (cookies `HttpOnly`), una divergencia entre contrato y comportamiento que sería estructuralmente imposible en un servicio SOAP, donde el WSDL *es* el contrato ejecutable.

Segundo, la seguridad. Es habitual asumir que REST es automáticamente más seguro por usar HTTPS, pero eso confunde dos capas distintas: WS-Security protege el contenido del mensaje — firma y cifrado que viajan con el documento y sobreviven a cualquier intermediario — mientras que TLS protege únicamente el canal de transporte, y el mensaje queda en claro en cada punto donde se termina esa conexión (por ejemplo, en un balanceador de carga). Para una API web autenticada con JWT sobre HTTPS, como BIOPET, la protección de transporte es suficiente; para un documento con valor legal que deba conservar garantías de no repudio más allá de la conexión que lo transportó, no lo es.

Tercero, el rendimiento. La diferencia no se reduce al tamaño del mensaje. El factor más determinante es la cacheabilidad: en REST, un `GET` es seguro e idempotente por definición, por lo que proxies, CDN y cachés de aplicación pueden almacenar la respuesta usando la infraestructura estándar de HTTP (así opera BIOPET con su caché Redis sobre el listado de mascotas). En SOAP, toda operación viaja típicamente por `POST`, que HTTP no cachea, por lo que cualquier optimización de caché debe construirse explícitamente.

4.5. Ejemplo práctico: la misma operación en SOAP y en REST

Para hacer la comparación concreta, se muestra una misma operación — **consultar los datos de la mascota con identificador 15** — en ambos enfoques. El lado SOAP es una implementación hipotética con fines comparativos (BIOPET no expone una interfaz SOAP real); el lado REST es la implementación real de BIOPET.

4.5.1. Enfoque SOAP (hipotético, con fines comparativos)

POST /ws/mascotas HTTP/1.1

Host: biopet.ec

Content-Type: text/xml; charset=utf-8

SOAPAction: "http://biopet.ec/ws/ObtenerMascota"

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header>
    <wsse:Security
      xmlns:wsse="...oasis-200401-wss-wssecurity-secext-1.0.xsd">
      <wsse:UsernameToken>
        <wsse:Username>vet@biopet.ec</wsse:Username>
        <wsse:Password>...</wsse:Password>
```

```

    </wsse:UsernameToken>
  </wsse:Security>
</soap:Header>
<soap:Body>
  <ObtenerMascota xmlns="http://biopet.ec/ws">
    <mascotaId>15</mascotaId>
  </ObtenerMascota>
</soap:Body>
</soap:Envelope>

```

Respuesta exitosa (recorte de los campos relevantes):

HTTP/1.1 200 OK

Content-Type: text/xml; charset=utf-8

```

<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <ObtenerMascotaResponse xmlns="http://biopet.ec/ws">
      <mascota>
        <id>15</id><nombre>Luna</nombre><especie>Canino</especie>
        <raza>Labrador</raza>
        <fechaNacimiento>2021-03-14</fechaNacimiento>
      </mascota>
    </ObtenerMascotaResponse>
  </soap:Body>
</soap:Envelope>

```

Si la mascota no existe, la respuesta HTTP sigue siendo un 500 y el detalle del error viaja dentro del sobre:

```

<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Mascota no encontrada: 15</faultstring>
</soap:Fault>

```

4.5.2. Enfoque REST (implementación real de BIOPET)

GET /api/mascotas/15 HTTP/1.1

Host: localhost:8080

Accept: application/json

Cookie: access_token=eyJhbGciOiJIUzI1NiJ9...

Respuesta exitosa:

HTTP/1.1 200 OK

Content-Type: application/json

```

{
  "id": 15, "duenioId": 4, "duenioNombre": "Maria Perez",
  "nombre": "Luna", "especie": "Canino", "raza": "Labrador",
  "fechaNacimiento": "2021-03-14", "activo": true
}

```


Si la mascota no existe, el código de estado forma parte del contrato:

```
HTTP/1.1 404 Not Found
Content-Type: application/problem+json

{
  "type": "urn:biopet:error:not-found",
  "title": "Recurso no encontrado",
  "status": 404,
  "detail": "Mascota no encontrada: 15",
  "instance": "/api/mascotas/15"
}
```

Este cuerpo de error usa el formato `ProblemDetail` real de BIOPET (RFC 7807 (Nottingham & Wilde, 2016), citado así en el propio ADR-006 del proyecto; la IETF publicó en 2023 el RFC 9457 (Nottingham et al., 2023) como sucesor, que BIOPET todavía no adopta).

Tres diferencias resaltan de este ejemplo. Primera, el **volumen**: el mensaje SOAP ocupa varias veces el tamaño del JSON para transportar exactamente los mismos datos. Segunda, el **manejo de errores**: SOAP devuelve 500 incluso cuando el problema es del cliente (un identificador inexistente), obligando a interpretar el cuerpo para saber qué ocurrió realmente; REST usa el código de estado como primer nivel de información. Tercera, la **identificación del recurso**: en REST la mascota 15 tiene una URI propia y estable (`/api/mascotas/15`), enlazable y cacheable; en SOAP el identificador es un parámetro dentro de un cuerpo enviado siempre a la misma dirección, por lo que ninguna capa intermedia puede cachearlo.

Precisión aplicada a este ejemplo

El aporte original de Carvajal representaba el error 404 con `"type": "https://biopet.ec/problems/not-found"`. Al adaptarlo a este informe se corrigió ese valor por el real de BIOPET, `urn:biopet:error:not-found`, verificado directamente en `Backend/src/main/java/com/biopet/exception/ProblemType.java` y en `GlobalExceptionHandler`. El resto del cuerpo (`title`, `detail`, formato general) ya coincidía con el código real.

4.6. Casos de uso en el contexto ecuatoriano

Facturación electrónica del SRI — SOAP (caso sustentado). El esquema de comprobantes electrónicos del Servicio de Rentas Internas del Ecuador opera mediante servicios web SOAP descritos por WSDL (Servicio de Rentas Internas del Ecuador, s.f.): el contribuyente envía el comprobante en XML al servicio de recepción y luego consulta su autorización. La elección es coherente con el requisito: el comprobante debe llevar firma digital con valor tributario *por sí mismo*, independientemente del canal por el que viaje — exactamente lo que WS-Security resuelve y que TLS, al proteger solo el canal, no resuelve. A esto se suma que miles de emisores heterogéneos deben implementar un contrato idéntico y verificable, la fortaleza principal del WSDL.

Integraciones del sector financiero — SOAP (escenario aplicable, no atribuido a una entidad específica). En el sector financiero ecuatoriano es un escenario

razonable que la integración entre sistemas *core* bancarios, buses de servicios empresariales y validación de identidad se apoye en servicios SOAP, dadas la necesidad de trazabilidad y garantías transaccionales en operaciones que mueven dinero, y la convivencia con sistemas legados cuya reescritura no siempre se justifica. Este informe no afirma que una entidad bancaria concreta use SOAP: se presenta como un escenario de aplicación típico del sector, no como un hecho verificado sobre una institución particular.

Aplicaciones web y móviles de consumo, incluida BIOPET — REST. Plataformas de envío a domicilio, comercio electrónico, telemedicina y gestión clínica en Ecuador se construyen típicamente sobre APIs REST/JSON, porque el consumidor es un navegador o una app móvil, el payload debe ser liviano y el ciclo de cambios es rápido. BIOPET encaja en este perfil de forma verificable: frontend Angular, autenticación JWT, documentación Swagger y despliegue en contenedores (capítulos 2 y 3).

En la práctica, muchas organizaciones ecuatorianas operan en un modelo híbrido: mantienen integraciones SOAP hacia sistemas legados o regulatorios (como el SRI) y exponen simultáneamente una fachada REST hacia sus aplicaciones web y móviles. La pregunta relevante no es “¿SOAP o REST?”, sino en qué frontera de la arquitectura conviene cada uno.

4.7. Cuándo conviene cada enfoque

Ninguno de los dos estilos es superior en abstracto; la elección depende de los requisitos del dominio. **SOAP** resulta apropiado cuando el contrato debe ser formal y verificable por máquinas antes de escribir código, cuando se exige no repudio o firma a nivel de mensaje, cuando hay transacciones distribuidas entre varios servicios, o cuando el transporte no es necesariamente HTTP. **REST** resulta apropiado cuando el consumidor principal es un navegador o una aplicación móvil, cuando se prioriza la velocidad de desarrollo y una curva de adopción baja para terceros, cuando conviene aprovechar la caché de HTTP, o cuando el sistema debe escalar horizontalmente sin estado compartido.

Para BIOPET, REST es la elección adecuada: el consumidor es un frontend Angular que se beneficia de JSON nativo, la autenticación con JWT y cookies encaja en el modelo *stateless* que REST exige, y la caché en Redis se apoya en que los GET son seguros e idempotentes. Ninguno de los requisitos del dominio veterinario del sistema — agendar citas, registrar consultas, controlar vacunas — exige no repudio, transaccionalidad distribuida entre servicios ni transporte fuera de HTTP.

Capítulo 5

Tendencias web actuales

5.1. Propósito de este análisis

La actividad de Unidad IV exige, además de la revisión cruzada del PFC seleccionado, una investigación sobre tendencias contemporáneas del desarrollo web. Este capítulo analiza tres de ellas — Jamstack, *Progressive Web Apps* (PWA) e inteligencia artificial generativa — y las contrasta con el estado real de BIOPET (Angular 17.3, Spring Boot 3.2.12/Java 21, PostgreSQL 16, Redis 7; capítulos 2 y 3), de modo que la reflexión no quede en el plano puramente teórico. La investigación y redacción original corresponde a Fajardo Montes Michael Xavier, integrante del Equipo H; su aporte es de investigación y análisis documental, no de implementación: Fajardo no participó en el desarrollo del código de BIOPET, ni en la construcción del frontend Angular o del backend Spring Boot (ver sección 1.2). Este capítulo adapta y reorganiza ese aporte al formato del informe, integrando sus citas bibliográficas verificables.

5.2. Jamstack

Jamstack — acrónimo de *JavaScript, APIs y Markup* — no es un *framework* sino un **patrón arquitectónico**: un conjunto de decisiones de diseño orientadas a desacoplar el frontend del backend (Jamstack.org, 2026). En una aplicación Jamstack, el contenido se pre-construye durante el proceso de *build* en páginas y *assets* estáticos que se distribuyen directamente desde una CDN, mientras que toda funcionalidad dinámica — autenticación, formularios, búsqueda, pagos — se delega a APIs externas o propias consumidas desde el cliente (Cloudflare, 2026). Esta separación reduce la superficie de ataque, porque no hay un servidor de aplicación expuesto en cada petición, y facilita el escalado horizontal, porque servir archivos estáticos desde una CDN es una operación mucho más barata que ejecutar lógica de negocio en cada *request*.

Jamstack también tiene limitaciones reales: al delegar lo dinámico a servicios de terceros, la aplicación hereda la disponibilidad de esos proveedores, y al no existir una única forma “correcta” de componer el stack, cada proyecto combina herramientas distintas, lo que puede elevar la curva de aprendizaje frente a un monolito tradicional. Por eso tiene sentido para blogs, *landing pages*, documentación técnica, sitios de marketing y catálogos sin lógica de servidor compleja por petición; no es el enfoque más natural para sistemas altamente transaccionales, con datos que cambian constantemente por usuario y lógica de autorización fina.

BIOPET no es, y no debería forzarse a ser, una aplicación Jamstack. El sistema es intrínsecamente transaccional: cada rol (ADMIN, VETERINARIO, AUXILIAR, DUENO) ve datos distintos, el rol DUENO está limitado a sus propias mascotas, la autenticación depende de cookies de sesión validadas contra Redis en cada petición, y hay operaciones de escritura constantes (alta, edición y baja lógica de mascotas). Pre-renderizar contenido estático y servirlo desde una CDN no aporta valor cuando casi todo el contenido depende del usuario autenticado y cambia con frecuencia: una migración completa a Jamstack solo trasladaría complejidad del backend al cliente, sin beneficio real sobre la arquitectura actual, basada en un backend Spring Boot con persistencia PostgreSQL (capítulo 3).

5.3. Progressive Web Apps (PWA)

Según la documentación oficial de MDN Web Docs, una PWA es una aplicación construida con tecnologías estándar de la plataforma web que ofrece una experiencia cercana a la de una aplicación nativa, mediante un *manifesto web* (`manifest.json`) que declara ícono, pantalla de bienvenida, color de tema y modo de visualización, y que permite al navegador instalarla en el dispositivo del usuario (MDN Web Docs, 2026). El componente central que habilita ese comportamiento tipo app es el *service worker*: un script que se ejecuta en un hilo separado del JavaScript principal de la página y actúa como proxy entre el navegador y la red, capaz de interceptar peticiones y cachear *assets* para ponerlos disponibles cuando la conectividad falla (MDN Web Docs, 2025).

Conviene precisar que una PWA **no funciona completamente sin conexión por definición**: el alcance real del trabajo offline depende de qué recursos decide cachear explícitamente el service worker configurado, y de qué operaciones tienen sentido sin conectar con el servidor. Una PWA bien diseñada puede ofrecer lectura de contenido previamente visitado y una carga más rápida incluso con red inestable, pero no puede, por sí sola, garantizar que operaciones de escritura contra un backend remoto se completen sin conexión real.

BIOPET actualmente no implementa PWA. La revisión del frontend confirma que no existe un `manifest` de tipo PWA, ni configuración `ngsw` (Angular service worker), ni las dependencias `@angular/pwa` o `@angular/service-worker` en `frontend/package.json`. Es, sin embargo, la tendencia con mayor valor concreto como evolución futura: el frontend ya es una SPA en Angular 17.3, punto de partida técnico más simple para agregar `manifest` y `service worker` sin rehacer la arquitectura. Funciones razonables que podrían beneficiarse — sin que estén implementadas hoy — incluyen consultar mascotas ya visualizadas con anterioridad, un acceso más rápido desde dispositivos móviles del personal clínico, mayor tolerancia a conexiones inestables durante visitas a domicilio, y una caché selectiva de recursos no sensibles, evitando siempre cachear información clínica sensible fuera del control del backend.

5.4. Inteligencia artificial generativa en el desarrollo web

El uso de modelos generativos como apoyo en tareas de programación — generación de código repetitivo (*boilerplate*), refactorización, redacción de pruebas unitarias, explicación de código heredado y generación de documentación técnica — se ha extendido en los últimos años como práctica habitual en equipos de desarrollo. Una revisión académica reciente, que

analizó 87 publicaciones sobre el tema, identifica aportes de estos modelos en seis etapas del ciclo de vida del software: ingeniería de requisitos, diseño arquitectónico, implementación, aseguramiento de calidad, mantenimiento y automatización de DevOps (Arsha & Sreeji, 2026).

Esa misma adopción trae riesgos documentados que no deben minimizarse. El informe 2026 de Veracode sobre seguridad del código generado por IA, tras evaluar más de cien modelos en tareas de Java, JavaScript, Python y C#, encontró que el 45 % del código generado introdujo al menos una vulnerabilidad del OWASP Top 10, con Java como el lenguaje de mayor riesgo (Veracode, 2026). Un modelo generativo puede sugerir patrones plausibles sin que eso equivalga a un criterio de ingeniería: no sopesa una hoja de ruta a mediano plazo contra el stack de habilidades del equipo, los requisitos de cumplimiento normativo, ni los modos de falla específicos de un sistema en producción. La conclusión práctica es que ninguna de estas herramientas debería adoptarse “porque está de moda”: acelera tareas repetitivas, pero introduce riesgo de seguridad medible si se integra sin revisión humana ni pruebas automatizadas que lo respalden.

5.4.1. Privacidad, propiedad intelectual y sesgos

Tres aspectos adicionales merecen una reflexión breve, porque suelen quedar fuera de las discusiones centradas únicamente en productividad. **Privacidad:** no deberían introducirse datos clínicos o personales reales en herramientas de IA generativa externas sin garantías contractuales y técnicas adecuadas sobre cómo se procesan y retienen esos datos; esto es directamente relevante para BIOPET, que maneja información de usuarios (dueños, personal clínico) y datos veterinarios asociados a mascotas identificables. **Propiedad intelectual:** quien integra código sugerido por un modelo generativo debe revisar, en la medida de lo posible, la procedencia y las condiciones de uso de ese código, dado que el origen y la licencia del material con el que fue entrenado el modelo no siempre son verificables desde el lado del usuario. **Sesgos:** las respuestas de un modelo generativo pueden reproducir patrones incorrectos o sesgados presentes en sus datos de entrenamiento — por ejemplo, convenciones de diseño obsoletas o inseguras que el modelo reproduce por su frecuencia histórica — por lo que la revisión humana no es un paso opcional sino una responsabilidad profesional ineludible antes de integrar cualquier sugerencia generada automáticamente, especialmente en módulos de autenticación y autorización.

5.5. Aplicación potencial en BIOPET

5.5.1. Estado actual

BIOPET es, de forma verificable, una *single-page application* en Angular 17.3 que consume una API REST documentada con OpenAPI/Swagger, sobre un backend Spring Boot 3.2.12/Java 21 con persistencia en PostgreSQL 16 y caché en Redis 7. La autenticación se resuelve mediante JWT entregado en cookies `HttpOnly/Secure`, con cuatro roles reales (ADMIN, VETERINARIO, AUXILIAR, DUENO) y control de acceso por rol y por propiedad sobre el recurso `mascotas` (capítulos 2 y 3). Ninguna de las tres tendencias analizadas en este capítulo — Jamstack, PWA, IA generativa — forma parte de esta arquitectura actual.

5.5.2. Evolución posible

De las tres tendencias, **PWA** es la que representa una mejora razonable para BIOPET: aprovecha que el frontend ya es una SPA en Angular, no exige rehacer la arquitectura de seguridad del backend, y resuelve un problema concreto (acceso a información de mascotas con conectividad intermitente en visitas a domicilio o zonas con señal débil). **La IA generativa** puede aportar como apoyo al desarrollo — generación de casos de prueba adicionales, documentación de la API, resúmenes de cambios — siempre bajo revisión humana estricta, con especial cautela en módulos de autorización por rol y propiedad, donde la evidencia citada en la sección 5.4 muestra que el código generado falla con mayor frecuencia en seguridad. **Jamstack**, en cambio, resulta menos apropiado para el núcleo transaccional y multirol de BIOPET; distinto sería el caso de contenido público o estático ajeno a ese núcleo — por ejemplo, una futura página informativa de la clínica sin datos de usuario — que sí podría beneficiarse de técnicas de pre-renderizado, sin migrar el sistema completo.

No se propone, en ningún caso, migrar BIOPET a Jamstack ni presentarlo como si ya implementara PWA: ambas siguen siendo, respectivamente, un enfoque poco adecuado para el núcleo del sistema y una evolución futura todavía no implementada.

Capítulo 6

Revisión cruzada del PFC

6.1. Metodología de revisión cruzada

Como parte de la actividad de Unidad IV, dos integrantes del Equipo H que no participaron en el desarrollo original de BIOPET realizaron revisiones críticas independientes del proyecto: Carvajal Loor Johan Stalin, con enfoque en arquitectura MVC, backend, contrato OpenAPI, seguridad, base de datos, caché y mantenibilidad (docs/u4/revisiones/REVISION-CARVAJAL.md), y Fajardo Montes Michael Xavier, con enfoque en usabilidad, escalabilidad, pruebas, rendimiento, documentación y despliegue (docs/u4/revisiones/REVISION-FAJARDO.md). Ambas revisiones se redactaron por separado, sobre el mismo estado del repositorio, antes de consolidarse en este capítulo. El PFC BIOPET fue desarrollado por su equipo original — Mariscal Cabrera Jaime Josué, Beltrán Montiel Fred Adrián y Taipe Mora Zaida Melissa —; Carvajal y Fajardo no participaron en su implementación: su aporte a esta actividad es de revisión crítica e investigación documental sobre un sistema ya construido (ver sección 1.2). Este capítulo no redacta una tercera revisión independiente ni sustituye a las dos originales: las sintetiza, contrasta sus coincidencias y complementos, y las contrasta con el estado actual del repositorio.

6.2. Fortalezas identificadas

Ambas revisiones coinciden en que BIOPET sostiene una disciplina de ingeniería poco habitual en un proyecto de curso. **La separación en capas** (*controller* → *service* → *repository* → *entity*) no se rompe en ningún controlador, y las entidades JPA nunca se exponen directamente: se serializan como DTOs `record` de Java 21. **El control de acceso opera en dos niveles complementarios**: el rol se valida de forma declarativa con `@PreAuthorize`, y la propiedad del recurso se verifica además en el servicio, de modo que un `DUENO` no puede, en la mayoría de los endpoints, leer el registro de otro (sección 6.3.1 matiza una excepción real a esta regla). **La autenticación JWT** se entrega en cookies `HttpOnly/Secure/SameSite=Strict`, sin uso de `localStorage` en el frontend Angular, con revocación efectiva en Redis. **Los errores se estandarizan con `ProblemDetail`** (RFC 7807) de forma consistente en los seis controladores. **La cobertura de pruebas es alta y verificada automáticamente**: 166 pruebas ejecutadas, 0 fallos y 0 errores, sobre 45 clases analizadas por JaCoCo, con un umbral de `jacoco:check` que falla el build por debajo del 60 % en LINE, BRANCH y COMPLEXITY (docs/mediciones/sec/jacoco-summary.md), y con pruebas orientadas a riesgo (inyección SQL, cabeceras de seguridad, disponibilidad de Swagger) y no solo a llenar

cifras. **Redis se usa con criterio** tanto para caché de lecturas frecuentes como para la lista negra de tokens revocados. Finalmente, **la reproducibilidad del entorno** — imágenes fijadas por digest `sha256`, `healthcheck` encadenado entre servicios, ADRs numerados (ADR-002 a ADR-007) — permitió que ambas revisiones levantaran el sistema desde cero sin intervención manual, condición necesaria para que la revisión cruzada fuera posible.

6.3. Debilidades y riesgos

Las debilidades detectadas no comprometen el funcionamiento actual del sistema, pero señalan brechas entre lo implementado y lo documentado, o entre el diseño y el comportamiento bajo condiciones que la evaluación académica no ejerció. **El esquema de base de datos tiene tres fuentes y una ya divergió:** junto a las migraciones de Flyway (única fuente que realmente ejecuta el sistema) coexisten `db/schema.sql` — desactualizado, sin reflejar las migraciones V2 a V4 — y `database/migrations/`, una copia huérfana sin referencias en el proyecto. **El contrato OpenAPI es estructural pero no descriptivo:** no existe una sola anotación `@Operation` ni `@ApiResponse` en todo el backend, la versión declarada (`v0.1.0`) no coincide con la del repositorio (`v0.9.0-rc`), y el esquema de seguridad publicado solo declara `bearerAuth`, sin describir la cookie `HttpOnly` que realmente usa el frontend. **Hay secretos con valores por defecto versionados en el código fuente:** si la variable `JWT_SECRET` no se define, la aplicación arranca igual firmando tokens con una clave publicada en el repositorio, y la contraseña de la cuenta administradora sembrada (`admin@biopet.ec`) está escrita directamente en `DataInitializer.java` y en `db/seed.sql`, aunque el propio README advierte que debe externalizarse antes de cualquier despliegue fuera del entorno académico. **La integración con la API externa carece de prueba automatizada dedicada:** no existe `ExternalApiServiceTest` ni `ExternalApiControllerTest`, pese a ser la pieza con más caminos de fallo del sistema (caché vacía, JSON corrupto, tercero caído, *timeout*). **El limitador de intentos de login es en memoria y no distribuido** (`ConcurrentHashMap` por instancia del backend), por lo que dejaría de funcionar correctamente si el sistema se desplegara con más de una instancia detrás de un balanceador. **Los registros de auditoría (AUTH_AUDIT) quedan locales al proceso/contenedor**, sin integración con un SIEM centralizado. **La cobertura de ramas condicionales sigue siendo notablemente menor que la de líneas:** 67.98 % en BRANCH frente a 87.45 % en LINE (71.81 % en COMPLEXITY), lo que sugiere casos de error y combinaciones de rol todavía no ejercitados por las pruebas, aunque ambas cifras superan holgadamente el umbral mínimo del 60 %. **La evidencia Lighthouse disponible no cubre aún la vista autenticada de Mascotas:** existen 6 corridas oficiales versionadas (`docs/mediciones/lighthouse/README.md`), con Performance, Accessibility y Best Practices dentro de umbral, pero SEO obtuvo 82 frente al umbral configurado de 90, y la corrida sobre `/mascotas` termina auditando `/login` porque el `authGuard` redirige sin sesión activa. Por último, el frontend no cuenta aún con pruebas *end-to-end* propias ni con las mejoras de accesibilidad móvil discutidas en el capítulo 5 (PWA sigue siendo, allí, una evolución propuesta y no implementada).

6.3.1. Inconsistencia de autorización en el listado de consultas

Al contrastar ambas revisiones con el estado actual del código surgió un hallazgo adicional, no señalado por ninguna de las dos revisiones originales, que conviene documentar con prudencia antes de integrar el capítulo de conclusiones. `GET /api/consultas`

admite el rol `ROLE_DUENO` (`ConsultaController.java:25-26`), pero el listado que expone `ConsultaService.listar()` (`ConsultaService.java:35-45`) no aplica ningún filtrado por propietario para ese rol: la rama específica para `ROLE_DUENO` ejecuta la misma consulta sin restricciones que la rama de personal clínico, pese a que el propio comentario del código indica que el “filtrado real de propiedad se aplica en `buscar()`”. En efecto, `GET /api/consultas/{id}` sí ejecuta control de acceso individual mediante `verificarAcceso()` (`ConsultaService.java:124-132`), que compara el dueño de la mascota asociada con el usuario autenticado. Esta diferencia entre el endpoint de listado y el de detalle constituye una **inconsistencia actual de autorización**: un usuario con rol `DUENO` que consulte `GET /api/consultas` puede recibir, en el listado paginado, consultas clínicas — diagnóstico, tratamiento, observaciones — de mascotas que no le pertenecen, aunque el acceso individual a cada una de ellas por `id` sí esté correctamente restringido. El riesgo es de **exposición de información clínica entre propietarios distintos**, un dato sensible en un sistema veterinario, y se clasifica como **mejora prioritaria** en la sección 6.5. Este informe no modifica el código del PFC: el hallazgo se deja documentado para que el equipo original decida cómo corregirlo (por ejemplo, replicando en `listar()` el mismo filtro por dueño que ya usa `MascotaService` para el recurso análogo de mascotas).

6.4. Comparación de las dos revisiones

Las dos revisiones no compiten entre sí: cubren capas distintas del mismo sistema y, juntas, dan una imagen más completa que cualquiera de las dos por separado. La primera revisión aportó mayor profundidad de bajo nivel sobre el backend y su configuración — lectura línea a línea de controladores, servicios y archivos de configuración — y por eso concentra los hallazgos más técnicos: el desajuste del contrato OpenAPI, la estrategia de claves de caché, el detalle de las expresiones de autorización repetidas, y la divergencia de fuentes del esquema de base de datos. La segunda revisión aportó una mirada más transversal, centrada en qué tan verificable es cada afirmación del proyecto frente a evidencia externa medible: usabilidad (SUS), cobertura de pruebas y su umbral automatizado, escalabilidad del rate limiting, superficie de seguridad expuesta en el README, y estado real de las mediciones de rendimiento (k6, Lighthouse). El contraste entre ambas evidenció que coinciden en dos ejes de fondo — la fortaleza del modelo de autenticación por cookies y la existencia de deuda técnica en secretos versionados por defecto — y que se complementan sin superponerse en casi todo lo demás: donde una revisión profundiza en cómo está construido un mecanismo, la otra verifica si existe evidencia de que ese mecanismo realmente funciona bajo medición.

6.5. Priorización de mejoras

La tabla 6.1 consolida las mejoras identificadas por ambas revisiones y por el contraste con el estado actual del repositorio, priorizadas por impacto real y no por facilidad de implementación.

Cuadro 6.1: Priorización de mejoras identificadas en la revisión cruzada.

Prioridad	Mejora	Justificación
Alta	Filtrar <code>GET /api/consultas</code> por propiedad para <code>ROLE_DUEÑO</code>	Cierra la exposición de información clínica de mascotas ajenas descrita en la sección 6.3.1
Alta	Eliminar los valores por defecto de <code>JWT_SECRET</code> y de la contraseña del admin sembrado	Evita falsificación de tokens y acceso administrativo no autorizado ante un fallo de configuración
Alta	Añadir prueba automatizada dedicada a la integración con la API externa	Es la única pieza funcional sin cobertura, con varios caminos de fallo no verificados
Media	Migrar el limitador de intentos de login a Redis	Sostiene el límite de intentos fallidos si el sistema llega a escalar a más de una instancia
Media	Anotar la API con <code>@Operation/@ApiResponse</code> s y declarar el esquema de autenticación por cookie en OpenAPI	Hace que el contrato publicado describa el comportamiento real del sistema
Media	Reforzar la cobertura <code>BRANCH</code> en rutas de error y autorización por rol/propiedad	Reduce el riesgo de casos borde no verificados pese a superar ya el umbral mínimo
Baja	Unificar el esquema en Flyway y enlazar desde el README las mediciones SUS y Lighthouse ya versionadas	Mejora la navegabilidad y la visibilidad de evidencia que ya existe pero es difícil de ubicar

6.6. Síntesis de la revisión cruzada

Las dos revisiones independientes, junto con el contraste realizado en este capítulo contra el estado actual del repositorio, describen un mismo diagnóstico desde ángulos distintos: BIOPET es un PFC con una base de ingeniería sólida — arquitectura en capas disciplinada, control de acceso en dos niveles, autenticación por cookies bien diseñada, pruebas automatizadas con umbral y evidencia de medición real (JaCoCo, SUS, k6, Lighthouse) — sobre la que persisten brechas concretas y acotadas entre lo implementado, lo documentado y lo verificado bajo condiciones que la evaluación académica no sometió al sistema. Ninguna de esas brechas — incluida la inconsistencia de autorización en el listado de consultas — compromete hoy el funcionamiento del sistema para el alcance académico definido; todas quedan documentadas y priorizadas en este capítulo como insumo para el trabajo de mejora que corresponde a la propuesta de evolución futura del proyecto.

Capítulo 7

Resultados y evidencias

Este capítulo resume, con cifras verificadas contra el repositorio actual de BIOPET, la evidencia técnica empírica disponible al momento de esta revisión. Ninguna cifra de este capítulo fue estimada: todas provienen de los documentos de evidencia citados.

7.1. Pruebas automatizadas y cobertura (JaCoCo)

Cuadro 7.1: Resultado real de `mvn clean verify` (docs/mediciones/sec/jacoco-summary.md).

Métrica	Valor
Pruebas ejecutadas	166
Fallos / errores / omitidas	0 / 0 / 0
Clases analizadas por JaCoCo	45
Cobertura LINE	87.45 %
Cobertura BRANCH	67.98 %
Cobertura COMPLEXITY	71.81 %
Umbral automático (<code>jacoco:check</code> , regla BUNDLE)	≥ 60 % en las tres métricas
Resultado	BUILD SUCCESS

Las tres métricas configuradas superan el umbral mínimo del 60 %, con mayor margen en LINE que en BRANCH y COMPLEXITY — una cobertura alta de líneas no implica, por sí sola, que todas las ramas condicionales (casos de error, combinaciones de rol) estén ejercitadas con la misma profundidad.

7.2. Calidad web automatizada (Lighthouse)

Cuadro 7.2: Corridas representativas de Lighthouse (docs/mediciones/lighthouse/README.md), escala 0-100.

Solicitud	Performance	Accessibility	Best Practices	SEO
/login (directo)	94	91	100	82
/mascotas (redirige a /login sin sesión)	92	91	96	82

De los cuatro umbrales configurados (`lighthousec.js`: Performance ≥ 0.80 , Accessibility ≥ 0.90 , Best Practices ≥ 0.90 , SEO ≥ 0.90), tres se cumplen con margen; el de SEO no se alcanza (82 sobre 90) en ninguna de las 6 corridas oficiales. Es importante precisar el alcance real de esta evidencia: Lighthouse auditó directamente únicamente `/login`; la solicitud a `/mascotas` redirige a `/login` por no existir una sesión activa (comportamiento esperado del *guard* de autenticación), por lo que esta evidencia **no constituye todavía una auditoría de la vista autenticada de Mascotas**, y tampoco cubre pantallas incorporadas después de la fecha de la medición.

7.3. Usabilidad (SUS)

Según docs/mediciones/sus/REPORT.md, se aplicó el instrumento System Usability Scale de Brooke (1996) a $n = 10$ participantes externos al equipo del PFC, con una tarea común de onboarding (login, alta, edición y baja lógica de una mascota, cierre de sesión). El puntaje SUS promedio obtenido fue **74.75/100** (clasificación cualitativa: *Bueno*, escala de adjetivos de Bangor, Kortum y Miller), con un intervalo de confianza del 95 % de [57,82, 91,68].

7.4. Seguridad (OWASP)

El repositorio documenta evidencia real (código, pruebas automatizadas y capturas de ejecución contra el stack Docker real) para las categorías OWASP Top 10 (OWASP Foundation, 2021) A01 (control de acceso roto), A02 (fallos criptográficos / TLS), A03 (inyección), A05 (configuración de seguridad), A07 (fallos de identificación y autenticación) y A09 (registro y monitoreo de seguridad), en docs/mediciones/sec/. Esto incluye, entre otros controles: JWT firmado HMAC-SHA256 con los siete claims estándar de RFC 7519 (Jones et al., 2015), TLS 1.3 real verificado con `openssl s_client`, limitación de intentos de login (bloqueo tras 6 fallos consecutivos) y registro estructurado de eventos de autenticación sin datos sensibles.

7.5. Síntesis

En conjunto, la evidencia disponible muestra un backend con pruebas automatizadas extensas y cobertura verificada por umbral automático, una API REST documentada y probada de extremo a extremo, y evidencia de seguridad archivada para los controles

OWASP aplicables. Las áreas con evidencia parcial y explícitamente documentadas como tal son: la cobertura de rama/complexidad (por debajo del margen de LINE), el umbral de SEO en Lighthouse, y el alcance de la propia medición de Lighthouse (no cubre todas las pantallas del frontend).

Capítulo 8

Conclusiones

Las conclusiones de este informe se derivan directamente del trabajo realizado por el Equipo H durante la Unidad IV: la presentación y verificación técnica de BIOPET (capítulos 2 y 3), la investigación comparativa (capítulos 4 y 5), la revisión cruzada consolidada (capítulo 6) y la evidencia empírica recopilada (capítulo 7). No resumen esos capítulos: sintetizan lo que ese trabajo permite afirmar, y lo que todavía no permite afirmar, sobre el sistema revisado.

1. **La arquitectura por capas y el diseño REST facilitaron la revisión, sin que eso signifique que estén libres de fricciones.** La separación `controller` → `service` → `repository` que se verificó en los siete recursos del backend (capítulo 3) permitió razonar sobre cada capa por separado, documentar los endpoints de forma consistente y ejecutar pruebas de extremo a extremo con Postman/Newman sin necesidad de levantar el frontend. Esa misma revisión, sin embargo, también dejó ver que una arquitectura ordenada no elimina por sí sola inconsistencias puntuales de comportamiento entre recursos que deberían tratarse igual — la revisión cruzada documentó expresiones de autorización repetidas y diferencias de estrategia de caché entre controladores análogos (capítulo 6) —, lo que confirma que el patrón MVC ordena el código pero no sustituye a la disciplina de revisión caso por caso.
2. **La evidencia técnica disponible es real y verificable, no una declaración de intenciones.** El backend supera 166 pruebas automáticas sin fallos ni errores, con un umbral de cobertura (`jacoco:check`) que exige al menos 60 % en LINE, BRANCH y COMPLEXITY sobre 45 clases analizadas, y la documentación OpenAPI expuesta en `/api/docs` está respaldada por una prueba propia que confirma su disponibilidad real (capítulo 3). Esa misma evidencia, leída con cuidado, también marca sus propios límites: la cobertura de BRANCH (67.98 %) queda casi veinte puntos por debajo de la de LINE (87.45 %), y la medición de usabilidad con SUS (74.75/100, $n = 10$) corresponde a una muestra externa reducida, no a una validación a gran escala. Contar con evidencia no equivale a que esa evidencia agote todo lo que habría que medir.
3. **Ni SOAP ni REST resultaron superiores en abstracto: la elección depende del dominio.** La comparación del capítulo 4 mostró que REST es la opción adecuada para BIOPET porque su consumidor es un frontend web, prioriza velocidad de desarrollo y se beneficia de la cacheabilidad nativa de HTTP que el sistema efectivamente usa con Redis. Eso no convierte a SOAP en una tecnología obsoleta: en escenarios donde el contrato debe ser formal y verificable antes de escribir código, donde se exige seguridad a nivel de mensaje que sobreviva a intermediarios, o donde hay

integración empresarial con sistemas legados — como el propio caso de la facturación electrónica del SRI analizado en ese capítulo —, SOAP conserva ventajas concretas que REST no reproduce de forma nativa.

4. **De las tres tendencias web analizadas, ninguna debería adoptarse en BIOPET solo porque está de moda.** Jamstack no encaja bien con un sistema intrínsecamente transaccional, multirol y con contenido distinto por usuario autenticado (capítulo 5); una migración completa trasladaría complejidad del backend al cliente sin beneficio real. PWA sí aparece como una evolución razonable, porque el frontend ya es una SPA en Angular y el problema que resolvería — conectividad intermitente del personal clínico o de los dueños — es concreto, aunque hoy no esté implementada. La IA generativa puede apoyar tareas de desarrollo, documentación y pruebas, pero la evidencia citada en ese capítulo también mostró que el código que produce introduce vulnerabilidades con una frecuencia medible, por lo que su uso razonable exige revisión humana explícita y criterios de seguridad y privacidad, no adopción automática.
5. **Revisar el mismo proyecto desde dos perspectivas independientes sacó a la luz riesgos que no eran evidentes solo con observar que el sistema funcionaba.** La primera revisión profundizó en el backend y su configuración; la segunda verificó qué tan sostenida estaba cada afirmación del proyecto frente a evidencia medible (capítulo 6). El contraste entre ambas, además, permitió detectar un hallazgo que ninguna de las dos había señalado por separado: `GET /api/consultas` no filtra por propietario para el rol `DUENO`, a diferencia del acceso individual por `id`, que sí lo hace correctamente. Ese hallazgo quedó **documentado y priorizado, no corregido**: este informe no modifica código del PFC, y la decisión de cómo resolverlo corresponde al equipo original.
6. **Que un sistema funcione no significa que esté terminado ni libre de deuda técnica, y BIOPET no es una excepción a esa distinción.** El propio `README.md` del proyecto es explícito en que la versión actual no equivale a un sistema certificado para producción: usa un certificado TLS autofirmado, credenciales de desarrollo y no fue evaluado en un entorno equivalente a uno productivo (capítulo 2). A eso se suman los hallazgos de la revisión cruzada — secretos con valores por defecto versionados, un limitador de intentos de login que no está distribuido, un contrato OpenAPI que no describe el mecanismo de autenticación real que usa el frontend — y la inconsistencia de autorización en Consultas del punto anterior. Ninguno de esos aspectos impide que BIOPET cumpla el alcance académico definido para esta actividad; todos, sin embargo, distinguen con claridad un sistema que *funciona bajo las condiciones en que fue evaluado* de un sistema *completamente maduro y sin deuda técnica pendiente*, y esa distinción es, en sí misma, uno de los resultados más útiles de haber sometido el proyecto a una revisión cruzada real.

Capítulo 9

Trabajo futuro

Este capítulo traduce en propuestas concretas las limitaciones reales identificadas en la revisión cruzada (capítulo 6) y en la evidencia empírica del capítulo 7. No se trata de una lista de tecnologías de moda: cada propuesta responde a un hallazgo específico ya documentado sobre el estado actual de BIOPET, y ninguna de ellas está implementada todavía.

9.1. Prioridades inmediatas

9.1.1. Control de acceso en Consultas

La revisión cruzada confirmó que `GET /api/consultas` admite el rol `DUENO` pero no filtra el listado por propietario, mientras que la consulta individual por `id` sí aplica correctamente ese control (sección 6.3.1). Esta inconsistencia sigue **pendiente**: no fue corregida en el marco de esta GA, que no modifica código del PFC. El trabajo futuro consiste en aplicar al listado el mismo filtrado por propiedad que ya usa el endpoint individual, y agregar pruebas específicas que impidan explícitamente el acceso cruzado entre dueños distintos, de modo que una regresión futura sea detectada por la suite automática y no por una revisión manual.

9.1.2. Gestión de secretos

Actualmente `JWT_SECRET` y la contraseña de la cuenta administradora sembrada tienen valores por defecto versionados en el código fuente (sección 6.3). La mejora futura razonable no es diseñar un gestor de secretos empresarial, sino eliminar esos valores por defecto para que la aplicación falle explícitamente al arrancar si la variable de entorno correspondiente no está definida, y documentar qué mecanismo (variables de entorno por ambiente, o un gestor de secretos ligero si el proyecto llegara a desplegarse fuera del entorno académico) reemplazaría al valor hardcodeado, junto con una política mínima de rotación periódica de esos valores.

9.1.3. API externa

La integración con la API externa de especies funciona, pero carece de prueba automatizada dedicada (`ExternalApiServiceTest` o `ExternalApiControllerTest` no existen). El trabajo futuro incluye cubrir con pruebas los caminos de fallo ya identificados — *timeout*,

respuesta vacía, error del tercero, y fallo de escritura en Redis —, y evaluar una degradación controlada que sirva la última respuesta válida cacheada cuando el servicio externo no responda, en lugar de propagar siempre un error al usuario final.

9.2. Mejoras de calidad y escalabilidad

9.2.1. Rate limiting distribuido

El limitador de intentos de login actual mantiene su estado en memoria por instancia del backend. Esto no es un problema mientras el sistema corra como una sola instancia, tal como fue evaluado, pero cobra relevancia si BIOPET llegara a escalar horizontalmente: mover ese estado a Redis, que el proyecto ya usa para otros fines, permitiría que el límite de intentos se comparta entre instancias en lugar de reiniciarse en cada una.

9.2.2. Cobertura de ramas

La cobertura actual (LINE 87.45 %, BRANCH 67.98 %, COMPLEXITY 71.81 %, todas por encima del umbral mínimo del 60 %) es sólida pero desigual entre métricas. La meta razonable no es alcanzar el 100 % de cobertura, sino priorizar deliberadamente las ramas que hoy quedan menos ejercitadas: combinaciones de rol y propiedad en la capa de autorización, manejo de códigos de error (401/403/429), casos negativos de validación, y los caminos de fallo de la integración externa descritos en la sección 9.1.3.

9.2.3. OpenAPI

Swagger UI funciona correctamente hoy y está respaldado por una prueba propia (sección 3.3); lo que falta es que el contrato publicado describa el comportamiento real del sistema. El trabajo futuro incluye anotar las operaciones con `@Operation` y documentar las respuestas de error efectivamente implementadas, declarar el esquema de autenticación por cookie que realmente usa el frontend — hoy solo se declara `bearerAuth` —, y sincronizar la versión publicada en el contrato con la del proyecto.

9.3. Frontend y experiencia

9.3.1. Lighthouse

Las seis corridas oficiales de Lighthouse disponibles auditan efectivamente `/login`: la solicitud a `/mascotas` sin sesión activa redirige a esa misma pantalla, por lo que la vista autenticada de Mascotas todavía no tiene evidencia propia (sección 7.2). El trabajo futuro incluye ejecutar una auditoría con una sesión autenticada válida — por ejemplo inyectando la cookie de acceso antes de la medición — para obtener evidencia real de esa pantalla, y revisar la brecha de SEO (82 frente al umbral de 90) si las páginas públicas del sistema siguen siendo relevantes para ese criterio.

9.3.2. PWA

Como se analizó en el capítulo 5, convertir el frontend en una PWA es una evolución razonable — no una funcionalidad existente — que podría aportar instalación, caché

selectiva de recursos no sensibles, acceso a información de mascotas ya consultada, y resiliencia parcial ante conectividad inestable. Cualquier implementación futura debería evitar el almacenamiento offline indiscriminado de información clínica sensible, y no debería tratarse como prioridad mayor que las mejoras de seguridad y autorización descritas en las secciones anteriores.

9.4. Mantenibilidad documental

De forma breve, también queda pendiente: consolidar el esquema de base de datos en Flyway como única fuente de verdad y retirar los archivos divergentes o huérfanos (`db/schema.sql`, `database/migrations/`); enlazar desde el README las mediciones SUS y Lighthouse ya versionadas pero poco visibles; y mantener sincronizadas la versión del proyecto y la del contrato OpenAPI.

9.5. Priorización

Corto plazo control de acceso en Consultas (sección 9.1.1), gestión de secretos (sección 9.1.2), pruebas de la API externa (sección 9.1.3).

Mediano plazo rate limiting distribuido (sección 9.2.1), cobertura de ramas críticas (sección 9.2.2), OpenAPI descriptivo (sección 9.2.3).

Evolución posterior

auditorías Lighthouse autenticadas (sección 9.3.1), PWA (sección 9.3.2), y la mantenibilidad documental (sección 9.4).

Si en algún momento se explora IA generativa como apoyo al desarrollo de BIOPET, su uso razonable se limita a generación de pruebas adicionales, documentación técnica y asistencia en refactorización, siempre bajo revisión humana — nunca como IA clínica o diagnóstica, que queda fuera del alcance de esta actividad y del propio dominio verificado del sistema. Ninguna de las propuestas de este capítulo tiene fecha, costo ni responsable asignado: son líneas de mejora derivadas de hallazgos reales, no un cronograma de implementación.

Referencias

- Angular Team, Google. (s.f.). Angular Documentation. <https://angular.dev/>
- Arsha, C. H., & Sreeji, K. B. (2026). Generative AI in Software Engineering. *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*. <https://doi.org/10.22214/IJRASET.2026.77834>
- Brooke, J. (1996). SUS: A Quick and Dirty Usability Scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester & A. L. McClelland (Eds.), *Usability Evaluation in Industry*. Taylor & Francis.
- Brown, S. (s.f.). The C4 Model for Visualising Software Architecture. <https://c4model.com/>
- Cloudflare. (2026). What is JAMstack? <https://www.cloudflare.com/learning/performance/what-is-jamstack/>
- Fielding, R., Nottingham, M., & Reschke, J. (2022, junio). *HTTP Semantics* (RFC N.º 9110). IETF. <https://www.rfc-editor.org/rfc/rfc9110>
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Doctoral dissertation]. University of California, Irvine. <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Jamstack.org. (2026). What is the Jamstack? <https://jamstack.org/what-is-jamstack/>
- Jones, M., Bradley, J., & Sakimura, N. (2015, mayo). *JSON Web Token (JWT)* (RFC N.º 7519). IETF. <https://www.rfc-editor.org/rfc/rfc7519>
- MDN Web Docs. (2025). js13kGames: Making the PWA work offline with service workers. https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Tutorials/js13kGames/Offline_Service_workers
- MDN Web Docs. (2026). What is a progressive web app? https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/What_is_a_progressive_web_app
- Nottingham, M., & Wilde, E. (2016, marzo). *Problem Details for HTTP APIs* (RFC N.º 7807). IETF. <https://www.rfc-editor.org/rfc/rfc7807>
- Nottingham, M., Wilde, E., & Dalal, S. (2023, julio). *Problem Details for HTTP APIs* (RFC N.º 9457). IETF. <https://www.rfc-editor.org/rfc/rfc9457>
- OASIS. (2006). Web Services Security: SOAP Message Security 1.1. <https://docs.oasis-open.org/wss/v1.1/>
- OpenAPI Initiative. (s.f.). OpenAPI Specification. <https://spec.openapis.org/oas/latest.html>
- OWASP Foundation. (2021). OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks. <https://owasp.org/Top10/>
- Servicio de Rentas Internas del Ecuador. (s.f.). Ficha técnica de comprobantes electrónicos. <https://www.sri.gob.ec/facturacion-electronica>
- Spring Team. (s.f.). Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>

- Veracode. (2026). 2026 GenAI Code Security Report: AI Is Writing More of Your Code but Security Hasn't Caught Up. <https://www.veracode.com/blog/2026-genai-code-security-report-ai-risk/>
- World Wide Web Consortium (W3C). (s.f.). SOAP Version 1.2. <https://www.w3.org/TR/soap12/>
- World Wide Web Consortium (W3C). (2001). Web Services Description Language (WSDL) 1.1. <https://www.w3.org/TR/wsdl>